

Datasets and evaluation

Running the same four golden questions through both tools with an LLM judge.

Datasets and evaluation

Both hold a golden dataset and run your agent against it with an LLM judge.

The exact same four questions were loaded into both, imported from one Python list, so the comparison is fair.

langfuse

v4.6.0 OSS

llm-observability-lab / llm-lab / Datasets / llm-obs-lab-golden

+ New item

Upload CSV

Go to...

ctrl K

Home

Dashboards

Observability

Tracing

Sessions

Users

Monitors

Prompt Management

Prompts

Playground

Evaluation

Scores

Evaluators

Human Annotation

Datasets

Experiments

Star Langfuse

See the latest releases and help grow the community on GitHub

Langfuse

33k

Settings

Book a call

Support

admin

admin@jmgro...

Dataset llm-obs-lab-golden

ItemsExperiments

Search...

IDs

Filters

Columns

8/8

Item id	Source	Status	Created At	Input	Expected Output	Metadata	Actions
b6d92353-9d59-4...		Active	2026-08-07 11:15:28	{ "items": { "question": "weather in Mumbai, then multiply the temperature by 2" } }	{ "items": { "answer": "States Mumbai's current temperature from the weather tool and gives that temperature multiplied by 2." } }		
dc09870c-1e5e-4...		Active	2026-08-07 11:15:28	{ "items": { "question": "what do my docs say about retries?" } }	{ "items": { "answer": "Retry only idempotent operations, use exponential backoff with jitter, cap attempts at 3-5, retry on 429/5xx only" } }		
a0e8a271-4021-4...		Active	2026-08-07 11:15:28	{ "items": { "question": "what's the weather in Chennai and what's 100/7?" } }	{ "items": { "answer": "Chennai is 22°C and humid; 100/7 is approximately 14.29." } }		
8c64aa63-eec8-4...		Active	2026-08-07 11:15:28	{ "items": { "question": "what's 23*7?" } }	{ "items": { "answer": "161" } }		

Rows per page

50

Page 1 of 1

langfuse

v4.6.0 OSS

llm-observability-lab

llm-lab

Experiments

Go to...

CTRL K

Experiment

Baseline

baseline-gpt4o - 2026-08-07T05...

Experiment selection

Search experiments...

Display

Results

Analytics

My Views

Columns 9/12

Filters

Item Metadata

Metadata

Scores (categorical)

Scores (numeric)

Scores (boolean)

Trace Scores (categorical)

Trace Scores (numeric)

Trace Scores (boolean)

Input

question: "what's 23*7?"

Expected Output

answer: "161"

baseline-gpt4o - 2026-08-07T05:47:06.202042Z

Baseline

OUTPUT

"23 * 7 equals 161."

SCORES

correctness

1.00

METADATA

Item ID

8c64aa63-eec8-492b-9549-7d8c8b581bbe

Observation

0b85516cf67ac449

Execution Trace

ed09055d5e462fa81a8a7273dc944f7e

Total Cost

\$0.001645

Latency

6.96s

question: "what's the weather in chennai and what's 100/7?"

answer: "Chennai is 22°C and humid; 100/7 is approximately 14.29."

OUTPUT

"Error: weather service timeout for city='Chennai'"

SCORES

correctness

-

METADATA

Item ID

a0e8a271-4021-4c11-a300-f765329a1a3c

Observation

c21c8a4bcb3bbf6

Execution Trace

2daea7e1edec112cea63b4586cfea900

Total Cost

\$0.001153

Latency

5.97s

Rows per page

20

Page

1

of 1

<<

<

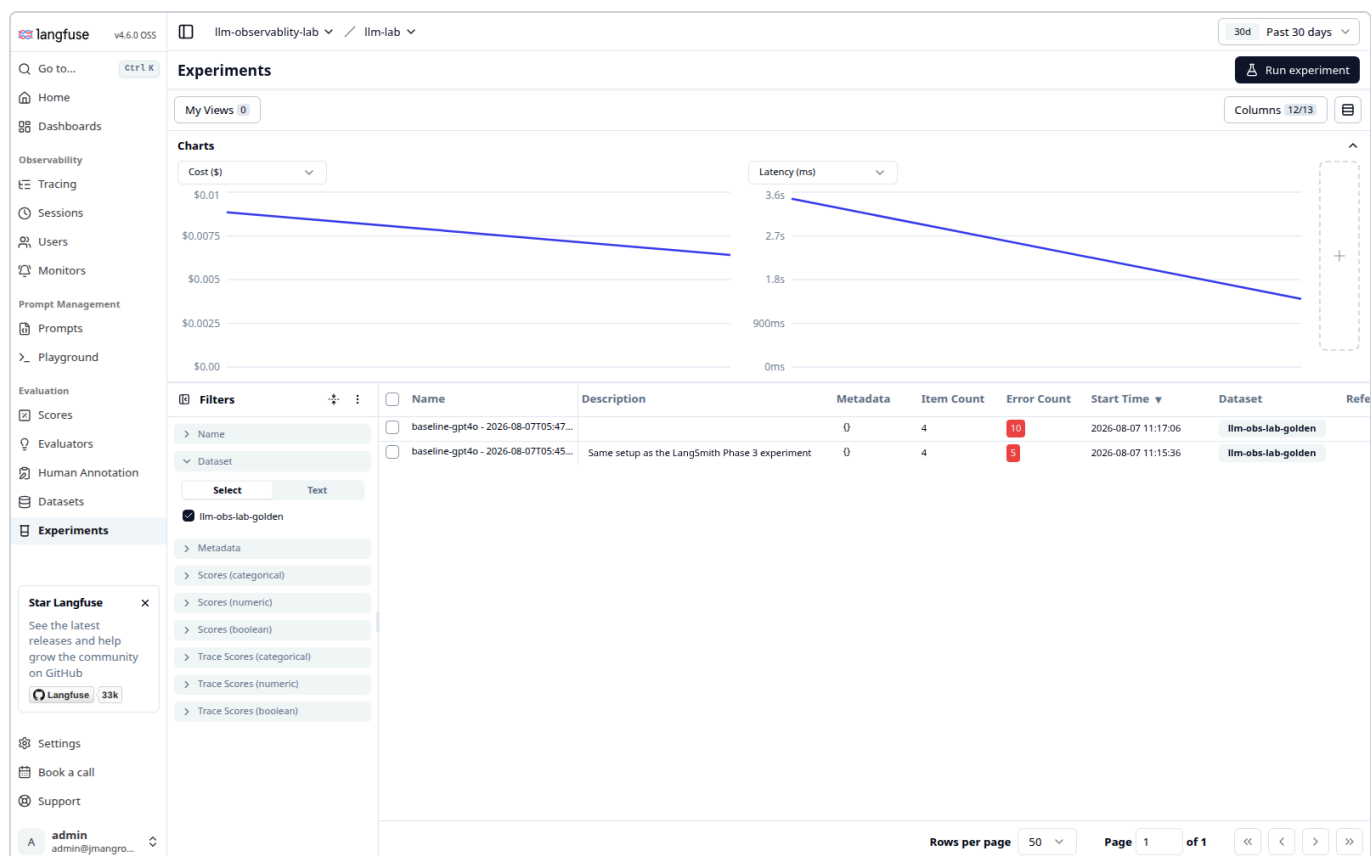
>

>>

Each row shows the question, the expected answer, what the agent actually said, the `correctness` score, cost, latency, and a link straight to the execution trace.

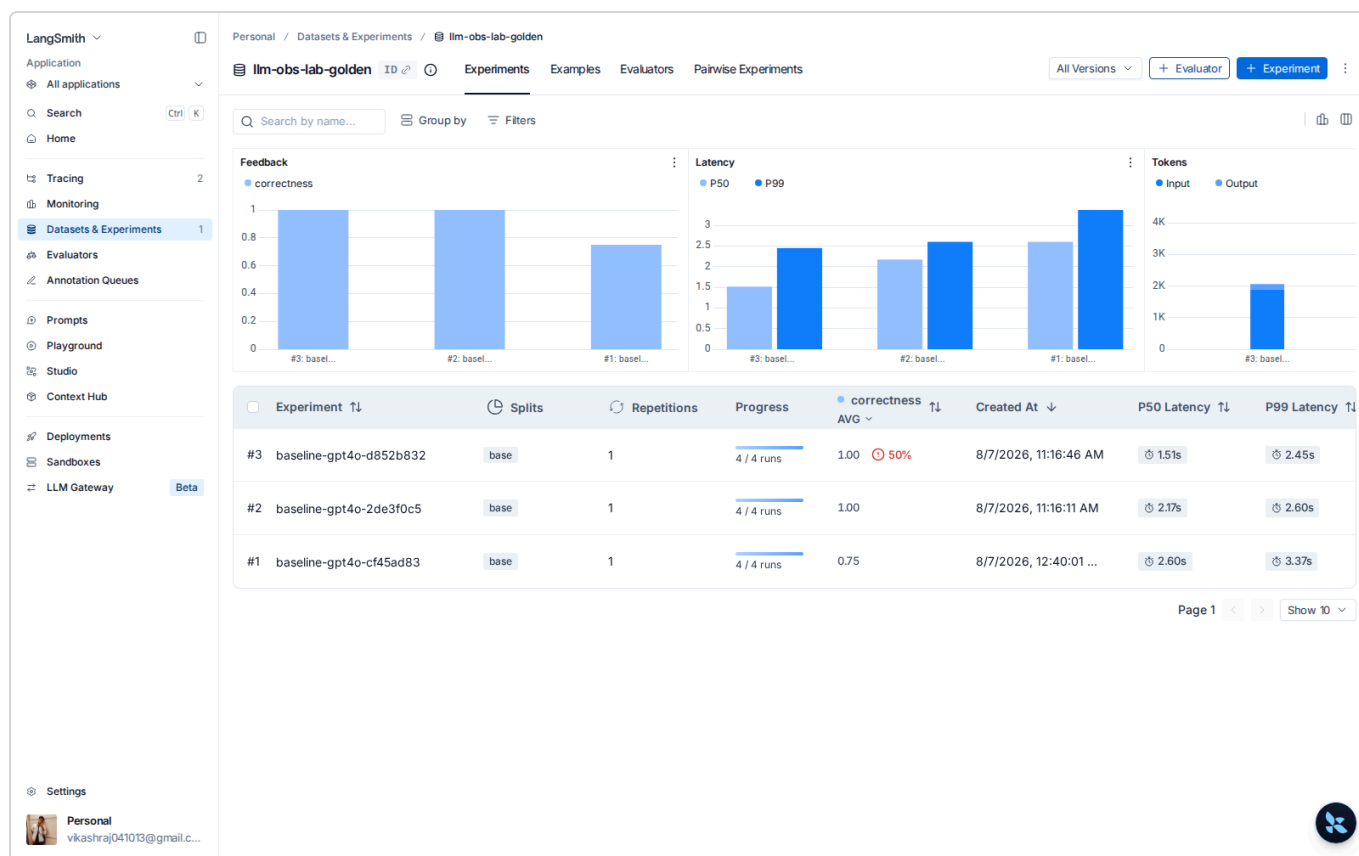
Note the second row. The agent output is `"Error: weather service timeout for city='Chennai'"` and the score is blank. **The failed case is still in the results.** It is not hidden.

And across runs:



Both runs show `Item Count 4` . The `Error Count` column (10 and 5) is how you spot a flaky agent at a glance.

LangSmith gives you the same thing with more charting on top:



Three experiments, each 4 / 4 runs, with average correctness of 1.00, 1.00 and 0.75, plus P50 and P99 latency per experiment. The red 50% badge on baseline-gpt4o-d852b832 is the forced-failure run. LangSmith surfaces error rate as a first class column next to the score, which is the fastest way to tell "my agent is wrong" apart from "my agent is broken".

The charts along the top (Feedback, Latency, Tokens) update as you add experiments, so regressions between runs are visible without opening anything.

The one real advantage Langfuse has here

Langfuse evaluators return an object with a comment field, so the judge explains itself:

```
return Evaluation(name="correctness", value=0.0, comment="Did not give the temperature.")
```

LangSmith evaluators return a bare bool. A 0.0 tells you it failed but not why. You can work around it, but Langfuse gives it to you by default.

Real numbers from the evaluation

Pulled from the LangSmith API, not typed by hand.

Clean run, everything succeeded:

Question	Score	Cost
weather in Mumbai, then multiply the temperature by 2	0.0	\$0.00278
what do my docs say about retries?	1.0	\$0.00290
what's 23*7?	1.0	\$0.00168
what's the weather in Chennai and what's 100/7?	1.0	\$0.00230

3 out of 4. The Mumbai case genuinely failed the judge. That is the eval doing its job.

Same dataset with the weather tool forced to fail every time:

Question	Result	Score
weather in Mumbai, then multiply the temperature by 2	ERROR	none
what do my docs say about retries?	ok	1.0
what's 23*7?	ok	1.0
what's the weather in Chennai and what's 100/7?	ERROR	none

Both tools behave the same way here, and this corrects a common assumption. Neither aborts the experiment. Both keep all 4 rows. Both leave the score empty on the failed ones.

The only difference is in Python. Langfuse drops failed items from the `result.item_results` list, so the script prints `experiment finished: 2 items` even though the UI correctly shows 4. If you log that number and never look at the UI, you will think your dataset shrank. LangSmith keeps all 4 in the returned results, but then runs your evaluator on the failed row too, which throws a second error unless you guard it:

```
answer=outputs.get("answer", "") # not outputs["answer"]
```